Listing 2.4 Backend code that uses OpenAI

```
import OpenAI from "openai";
import express from "express";
import cors from "cors";
import "dotenv/config";

const apiKey = process.env.OPENAI_API_KEY;
const openai = new OpenAI({ apiKey });
if (!apiKey) {
  throw new Error("Missing OPENAI_API_KEY environment variable");
}

const app = express();
app.use(express.json());
app.use(cors({ origin: "http://localhost:5173" }));
const openaiProvider = new OpenAIHandler(openai);

   app.post("/", (req, res) =>
openaiProvider.handleRequest(req, res));

app.listen(3000, () => {
  console.log("Started backend server");
});
```

Loads environment variables from an .env file

Retrieves the OPENAI_API_KEY from environment variables for authentication

Ensures the API key exists, throwing an error if not found

Parses incoming request bodies as JSON format

Enables cross-origin resource sharing to allow requests from the specified origin (likely the chat application frontend)

Creates an instance of the OpenAIHandler class

Defines a POST route handler that delegates the request to the handleRequest function within the openaiProvider instance

Starts the Express server on port 3000

The server.openai.js file sets up an Express server that listens for POST requests from the frontend, handles JSON payloads, and enables cross-origin resource sharing (CORS). The server authenticates with the OpenAI API using the provided API key and delegates incoming requests to the `OpenAIHandler` class. Within `OpenAIHandler`, the `handleRequest` method extracts the user's message from the request, sends it to the OpenAI `chat.completions` endpoint, and formats the assistant's response. Configuration options like `model`, `stop`, and `max_tokens` control which AI model is used, how the response is structured, and its length. Errors are logged and handled gracefully.

While the previous code established the server and API handlers, the heart of server communication lies within the `OpenAIHandler` class. This class, detailed in listing 2.5, is responsible for interacting with the OpenAI service endpoint (Class `OpenAIHandler` in server.js). As you do it, pay particular attention to the configuration values I provide. These values determine the specific OpenAI model used to generate responses (e.g., `gpt-3.5-turbo`) and how the initial system context is configured (e.g., the introductory prompt message).

Listing 2.5 Class that handles OpenAI requests

```
class OpenAIHandler {
  constructor(openai) {
    this.openai = openai;
  }

  async handleRequest(req, res) {
```

Stores OpenAI client instance; the constructor receives an OpenAI client instance as an argument and stores it for future use within the class.